

Tech Stack & Payment Integration Architecture · Step 5 of 5

Tech Stack & Payment Integration Architecture

Next.js + Supabase + Paystack/Flutterwave + Vercel, running entirely on free tiers: mobile money and Telecel Cash payments with webhook-driven instant delivery, multi-format files, bundle ZIP generation, signed download links, and a zero-cost launch checklist.

Date: August 25, 2026

Stack: Next.js · Supabase · Paystack · Vercel

Payments: MTN MoMo + Telecel Cash

Launch cost: GH¢ 0 fixed

Step 5 of 5 · Tech Stack & Payment Integration Architecture

Contents

- 1 · Tech Stack Recommendation (free tiers)
- 2 · Payment Integration Architecture
- 3 · Webhook Configuration Guide
- 4 · File Delivery Architecture
- 5 · API Route Specifications
- 6 · Database Schema
- 7 · Project Folder Structure
- 8 · File Storage & Security (signed URLs, MIME)
- 9 · Storage Usage Projection & Growth Plan
- 10 · Environment Variables
- 11 · Payment Resilience & Fallback Plan
- 12 · Zero-Cost Launch Checklist

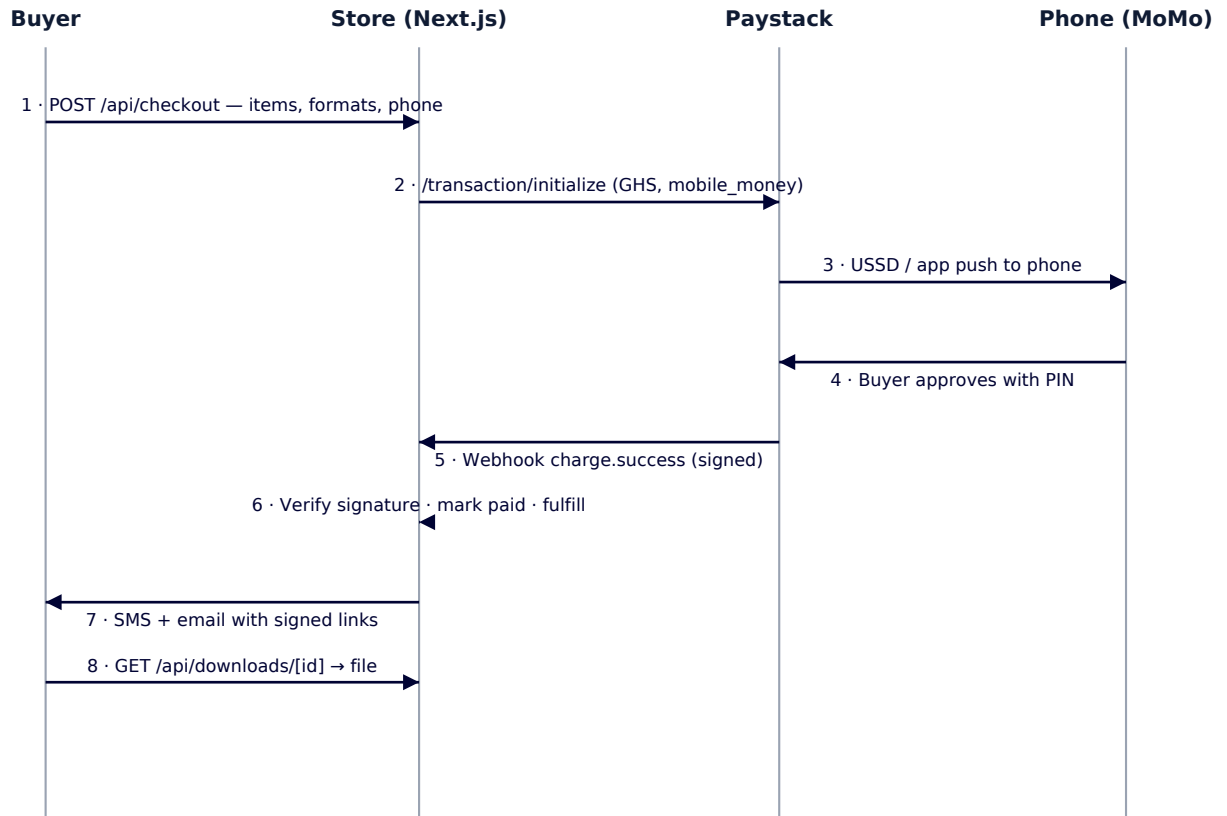
1 Tech Stack Recommendation

Everything runs on free tiers at launch. Payments are mobile-money only (no Stripe/PayPal) — the target audience pays with MTN MoMo and Telecel Cash.

Layer	Tool	Why this one	Free tier at launch
Frontend + API	Next.js 14+ (App Router, TypeScript)	One codebase for UI + API routes; server components; route handlers for webhooks; Vercel-native deploy.	Vercel Hobby: 1M function invocations/mo, 100 GB transfer, 60 s function timeout
Database + Auth	Supabase (Postgres)	Postgres with Row Level Security; built-in auth (email/OTP); realtime for status polling.	500 MB DB, 50K monthly active users, 2 projects
File storage	Supabase Storage	S3-compatible object storage with signed URLs, CDN, range requests, RLS on buckets.	1 GB storage, 5 GB egress/mo
Payments (primary)	Paystack (Ghana)	First-class Ghana MoMo: MTN MoMo, Telecel Cash (formerly Vodafone Cash), AirtelTigo via Charge API; reliable webhooks; T+1 settlement; ~1.95% capped GHS 100.	Free to integrate; per-transaction fee only
Payments (fallback)	Flutterwave (Ghana)	Second gateway for downtime failover; same channels (MTN, Telecel, AirtelTigo); switch via env flag.	Free to integrate; per-transaction fee only
Email	Resend	Transactional receipts + download links; React email templates; webhook-ready.	3,000 emails/mo free
SMS (optional)	Termii or Arkesal	Payment + download-link SMS to phone-first buyers. Only non-free item (~GH¢ 0.1–0.3/SMS).	Small pay-as-you-go; defer to post-launch
Background ZIP build	Next.js route + Supabase	Bundle ZIPs built once at publish time (admin action), stored, served via signed URL — avoids the 60 s serverless limit at checkout.	Uses existing function quota
Domain	Vercel subdomain (free) → custom later	Zero cost at launch; custom domain ~\$10/yr later.	Free *.vercel.app
Monitoring	Vercel dashboard + Supabase logs	Function logs, edge logs, DB query stats; no extra service needed.	Included

2 Payment Integration Architecture

Paystack (Ghana) is the primary gateway: it supports MTN MoMo, Telecel Cash (formerly Vodafone Cash) and AirtelTigo through one integration, with reliable webhooks. Flutterwave is the standby.



Step-by-step flow

Step 1 • Checkout → format selection

Buyer confirms per-item formats (PDF/EPUB/Word) and enters MTN or Telecel phone number. `POST /api/checkout` creates order (pending) + payment record.

Step 2 • Initialize charge

Server calls Paystack `POST /transaction/initialize` with amount (GHS), currency: GHS, channels: [mobile_money], and the buyer's phone. Paystack returns an authorization URL / access_code.

Step 3 • Mobile money prompt

For MoMo flows Paystack initiates a USSD/app push to the buyer's phone via the network (MTN MoMo / Telecel Cash). No card needed.

Step 4 • Buyer approves with PIN

Buyer approves the amount on their phone. The gateway marks the transaction as authorized.

Step 5 • Webhook fires

Paystack sends `POST charge.success` to `/api/webhooks/paystack` with the payment reference.

Step 6 • Verify signature + mark paid

Server verifies the x-paystack-signature (HMAC-SHA512 of the raw body). Marks payment paid, order fulfilled. Idempotent on reference.

Step 7 • Unlock downloads

Fulfillment job inserts downloads rows (per product × format), generates 48-hour signed URLs, and emails + SMSes the links.

Step 8 • Buyer downloads

GET /api/downloads/[id] validates the paid order, redirects to the signed URL. Re-download anytime from the dashboard.

Channel mapping (Paystack Charge API)

Buyer selects	Paystack channel	mobile_money.provider value	Notes
MTN Mobile Money	mobile_money	mtn	Phone prefixes 024/054/055/059
Telecel Cash	mobile_money	vodafone (Telecel)	Prefixes 020/026/027/050; rebranded from Vodafone Cash
AirtelTigo (future)	mobile_money	airteltigo	Prefixes 024/054/055/059 family

3 Webhook Configuration Guide

Paystack notifies the server the instant a payment succeeds; the webhook triggers verification and file delivery.

#	Step	Detail
1	Create the endpoint	POST /api/webhooks/paystack — Next.js route handler that reads the raw body (do not parse JSON before verifying).
2	Register in Paystack	Dashboard → Settings → API Keys & Webhooks → set Webhook URL to https://your-app.vercel.app/api/webhooks/paystack . Enable the charge.success event.
3	Verify signature	Header x-paystack-signature = HMAC-SHA512(secret, rawBody). Compare with crypto.timingSafeEqual. Reject mismatches with 401.
4	Handle only known events	Accept charge.success (and charge.failed for analytics). Ignore everything else with 200.
5	Idempotency	Look up payments.reference first. If the reference already exists with status=paid, return 200 immediately (retries are guaranteed by Paystack).
6	Respond fast	Return 200 within a few seconds. Do heavy work (ZIP, email) after the response or in a queued job — links are pre-built so fulfillment is quick.
7	Failure/retry behavior	Paystack retries webhooks for 3 days on failure. If still missed, admin triggers GET /transaction/verify/:reference (the verify endpoint) which returns the true status.
8	Flutterwave fallback	Same pattern: POST /api/webhooks/flutterwave, header Verif-Hash (HMAC-SHA256 of body with secret), event charge.completed. Selected by env var PAYMENT_GATEWAY=paystack flutterwave.

Reference implementation (Next.js route handler)

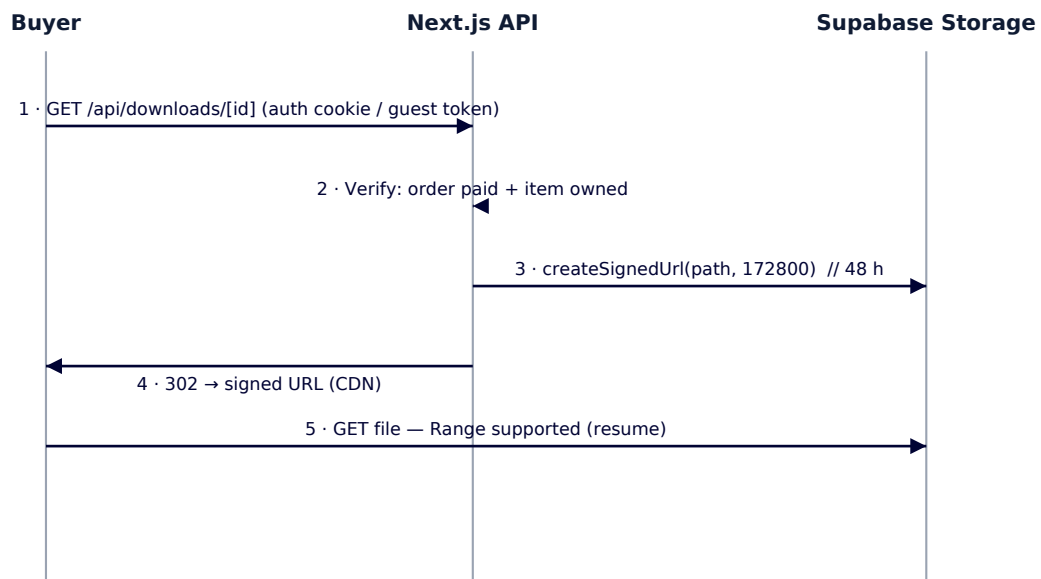
```
// app/api/webhooks/paystack/route.ts
import crypto from 'crypto';
import { verifyPayment, fulfillOrder } from '@lib/fulfillment';

export async function POST(req: Request) {
  const body = await req.text(); // RAW body – before JSON parsing
  const signature = req.headers.get('x-paystack-signature') ?? '';
  const expected = crypto
    .createHmac('sha512', process.env.PAYSTACK_WEBHOOK_SECRET!)
    .update(body)
    .digest('hex');

  if (!crypto.timingSafeEqual(Buffer.from(signature), Buffer.from(expected))) {
    return new Response('Invalid signature', { status: 401 });
  }
  const event = JSON.parse(body);
  if (event.event === 'charge.success') {
    await fulfillOrder(event.data.reference); // idempotent on reference
  }
  return new Response('OK', { status: 200 }); // acknowledge fast
}
```

4 File Delivery Architecture

Single-file downloads, format switching, and automatic bundle ZIP generation — all served through signed URLs.



Scenario	Flow	MIME type
Single prompt pack	GET /api/downloads/[id] → verify paid order + ownership → createSignedUrl(path, 172800) → 302 redirect → browser saves file. Word + PDF both linked.	application/pdf · application/vnd.openxmlformats-officedocument.wordprocessingml.document
Single ebook	Same flow; buyer chooses format at download time from format_options (PDF or EPUB). Each (order, product, format) is a row in downloads → format switching is free, forever.	application/pdf · application/epub+zip
Bundle ZIP	ZIP is pre-built at publish time (POST /api/admin/bundles/[id]/build-zip) and stored in bundle-zips/. Purchase unlocks one signed URL to the ZIP. If missing (edge), the route streams a fresh ZIP in chunks (fetch parts → archiver) — keep under the 60 s limit or use Vercel Fluid compute (300 s).	application/zip
Format switching	Dashboard shows every purchased product with all formats from format_options. Clicking generates a NEW signed URL — old links keep expiring.	—
Interrupted download	Supabase storage + CDN support HTTP Range requests, so partial downloads resume; the frontend retries with a fresh token.	—

MIME type map

Format	MIME type	Used for
PDF	application/pdf	Ebooks (print-ready), prompt packs
DOCX	application/vnd.openxmlformats-officedocument.wordprocessingml.document	Prompt packs (editable)

Format	MIME type	Used for
EPUB	application/epub+zip	Ebooks (reflowable, readers)
ZIP	application/zip	Bundles (all files in one archive)
JPG/PNG (covers)	image/jpeg · image/png	Cover images, sample screenshots

Bundle ZIP generation strategy

ZIPs are **built once at publish time** (admin triggers POST /api/admin/bundles/[id]/build-zip) and stored as a versioned object. This avoids assembling 20–150 MB archives inside the 60-second serverless window at checkout. A lazy fallback route streams a fresh ZIP in chunks if the stored one is ever missing. Component files are fetched from storage with range requests and piped through a streaming archiver (archiver/yazl).

5 API Route Specifications

All endpoints needed for storefront, payment, delivery, bundles, auth, and admin. Auth via Supabase session cookie; admin guarded by role claim.

Method	Endpoint	Auth	Purpose
GET	/api/products?type=prompt ebook bundle&category=&q=&sort=	public	Product listing with type filtering (storefront tabs), category filter, search, sort.
GET	/api/products/[id]	public	Product detail; returns type-specific fields (prompt_count, toc, format_options, sample_path).
GET	/api/products/[id]/sample	public	Sample prompts / free chapter preview.
POST	/api/lead-magnets	public	Email/phone capture for free samples; sends download link.
GET	/api/bundles/[id]	public	Bundle detail: bundle_items with individual product cards + savings summary.
GET	/api/cart	user or guest	Read cart (server-persisted for users, localStorage for guests).
POST	/api/cart	user or guest	Add item with selected format; validate product active.
PATCH	/api/cart/[id]	user or guest	Change format or quantity.
DELETE	/api/cart/[id]	user or guest	Remove item.
POST	/api/checkout	user or guest	Initiate: items + formats + email/phone + payment method (mtn_momo telecel_cash). Creates order (pending) + payment; returns authorization instructions.
GET	/api/orders/[id]	owner or admin	Status polling for the confirmation page.
GET	/api/orders/[id]/verify	owner	Fallback: server calls Paystack /transaction/verify/:reference and updates status.
POST	/api/webhooks/paystack	gateway	Payment confirmation webhook (charge.success).
POST	/api/webhooks/flutterwave	gateway	Fallback gateway webhook (charge.completed).
GET	/api/downloads	user (or guest token)	List all downloadable files for the account with formats.
POST	/api/downloads	user (or guest token)	Generate a fresh 48-h signed URL for (order, product, format).
GET	/api/downloads/[id]	owner	Validate access, log the download, 302 → signed URL.
GET	/api/bundles/[id]/zip	owner of paid order	Return signed URL to pre-built ZIP (or build lazily if missing).
POST	/api/auth/*	Supabase	Signup, login, OTP, password reset, callback (Supabase Auth handles).
GET/POST	/api/admin/products	admin	List/create prompts & ebooks.

Method	Endpoint	Auth	Purpose
PATCH/DELETE	/api/admin/products/[id]	admin	Update/archive product; re-validate files.
POST	/api/admin/bundles	admin	Create bundle from products; auto-compute value/savings; trigger ZIP build.
POST	/api/admin/bundles/[id]/build-zip	admin	Build/rebuild the bundle ZIP and store it.
GET	/api/admin/orders?status=&q=	admin	Order list with search by email/phone/reference.
PATCH	/api/admin/orders/[id]	admin	Update status (mark refunded, etc.).
POST	/api/admin/orders/[id]/resend	admin	Resend download links + email/SMS.
GET	/api/admin/analytics?range=7d	admin	Revenue by day/type/category, top products, conversion.
GET/PATCH	/api/admin/settings	admin	Gateway flag, link TTL, notification toggles.

6 Database Schema

Tables: users (Supabase Auth + profiles), products with product_type and format_options (formats[]), orders, payments, downloads with format tracking, and bundle_items linking bundles to components.

```
-- ===== SCHEMA: digital marketplace =====
-- users are managed by Supabase Auth (auth.users).
-- public.profiles links auth users to store data.
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text unique not null,
  phone text,
  full_name text,
  created_at timestamptz default now()
);

-- products: one row per sellable item; product_type drives
-- the page template, fields, and fulfillment logic.
create table public.products (
  id uuid primary key default gen_random_uuid(),
  sku text unique not null, -- PKG-FRE-01, EBK-MKT-01, BND-...
  title text not null,
  description text not null,
  product_type text not null check (product_type in ('prompt','ebook','bundle')),
  category text not null, -- freelance | marketing | smb | creators | dev
  price_ghs numeric(10,2) not null,
  -- type-specific fields (null where not applicable)
  prompt_count int,
  page_count int,
  author text,
  cover_path text, -- storage path, public bucket
  sample_path text, -- free chapter / sample prompts
  toc jsonb, -- ebook TOC (ordered list)
  formats text[] not null default '{}', -- format_options: {'pdf','docx'} | {'pdf','epub'} | {'zip'}
  file_paths jsonb not null default '{}', -- {pdf: 'product-files/ebooks/ebk-01.pdf', epub: '...'}
  active boolean not null default true,
  created_at timestamptz default now()
);

-- bundle_items: which products belong to which bundle.
create table public.bundle_items (
  id uuid primary key default gen_random_uuid(),
  bundle_id uuid not null references public.products(id) on delete cascade,
  product_id uuid not null references public.products(id) on delete cascade,
  position int not null default 0,
  unique (bundle_id, product_id)
);

create table public.orders (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id) on delete set null, -- null for guests
  email text not null,
  phone text,
```

```
status text not null default 'pending'
check (status in ('pending','paid','fulfilled','failed','refunded')),
total_ghs numeric(10,2) not null,
currency text not null default 'GHS',
created_at timestamptz default now()
);

create table public.payments (
id uuid primary key default gen_random_uuid(),
order_id uuid not null references public.orders(id) on delete cascade,
provider text not null check (provider in ('paystack','flutterwave')),
channel text not null check (channel in ('mtn_momo','telecel_cash','airteltigo','card')),
reference text unique not null, -- gateway reference (idempotency key)
status text not null default 'pending'
check (status in ('pending','authorized','paid','failed','refunded')),
amount_ghs numeric(10,2) not null,
raw jsonb, -- full webhook payload for audit
created_at timestamptz default now()
);

-- downloads: one row per (order, product, format) – enables
-- re-download and format switching with usage tracking.
create table public.downloads (
id uuid primary key default gen_random_uuid(),
order_id uuid not null references public.orders(id) on delete cascade,
product_id uuid not null references public.products(id) on delete cascade,
format text not null, -- pdf | docx | epub | zip
file_path text not null, -- storage path for this format
download_count int not null default 0,
last_downloaded_at timestamptz,
created_at timestamptz default now(),
unique (order_id, product_id, format)
);

-- lead magnet subscribers
create table public.subscribers (
id uuid primary key default gen_random_uuid(),
email text not null,
phone text,
source text not null default 'lead-magnet',
created_at timestamptz default now()
);

-- indexes for the hot queries
create index idx_products_type_active on public.products(product_type, active);
create index idx_orders_email on public.orders(email);
create index idx_payments_reference on public.payments(reference);
create index idx_downloads_order on public.downloads(order_id);
```

7 Project Folder Structure

A Next.js App Router project, ready to scaffold. All routes map 1:1 to the API spec in section 5.

```
my-digital-store/
├─ app/
│  ├─ (storefront)/
│  │  ├─ page.tsx # Home (hero, lead magnet, featured)
│  │  ├─ shop/page.tsx # Type tabs + category filter (searchParams)
│  │  ├─ products/
│  │  │  ├─ prompt-pack/[slug]/page.tsx
│  │  │  ├─ ebook/[slug]/page.tsx # cover, synopsis, TOC, author, format toggle
│  │  │  └─ bundle/[slug]/page.tsx # itemized items + strikethrough savings
│  │  ├─ free/[slug]/page.tsx # lead magnet capture
│  │  ├─ cart/page.tsx
│  │  ├─ checkout/page.tsx # step 1: contact + formats + method
│  │  ├─ checkout/pay/page.tsx # step 2: MoMo prompt + status polling
│  │  ├─ checkout/confirmation/[orderId]/page.tsx
│  │  ├─ about/page.tsx · faq/page.tsx · contact/page.tsx
│  │  └─ terms/page.tsx · privacy/page.tsx · refunds/page.tsx · delivery/page.tsx
│  ├─ (account)/
│  │  ├─ account/page.tsx # dashboard
│  │  ├─ account/downloads/page.tsx # re-download + format switching
│  │  ├─ account/orders/page.tsx # purchase history
│  │  └─ account/settings/page.tsx
│  └─ (admin)/
│  │  ├─ admin/page.tsx # KPI + revenue analytics
│  │  ├─ admin/products/page.tsx # list prompts & ebooks
│  │  ├─ admin/products/new/page.tsx # typed form (prompt vs ebook)
│  │  ├─ admin/bundles/page.tsx · admin/bundles/new/page.tsx
│  │  └─ admin/orders/page.tsx · admin/analytics/page.tsx · admin/settings/page.tsx
│  └─ api/
│  │  ├─ products/route.ts · products/[id]/route.ts · products/[id]/sample/route.ts
│  │  ├─ lead-magnets/route.ts
│  │  ├─ cart/route.ts · cart/[id]/route.ts
│  │  ├─ checkout/route.ts
│  │  ├─ orders/[id]/route.ts · orders/[id]/verify/route.ts
│  │  ├─ webhooks/paystack/route.ts · webhooks/flutterwave/route.ts
│  │  ├─ downloads/route.ts · downloads/[id]/route.ts
│  │  ├─ bundles/[id]/zip/route.ts
│  │  └─ admin/products/route.ts · admin/bundles/route.ts · admin/orders/route.ts
│  └─ admin/analytics/route.ts · admin/settings/route.ts
├─ lib/
│  ├─ supabase/client.ts · server.ts · admin.ts # anon / service-role clients
│  ├─ paystack.ts # initialize, verify, webhook signature check
│  ├─ flutterwave.ts # fallback gateway client
│  ├─ fulfillment.ts # mark paid → insert downloads → notify
│  ├─ storage.ts # signed URLs, MIME map, path builders
│  └─ zip.ts # bundle ZIP build (streaming)
├─ components/
│  ├─ products/ProductCard.tsx · PromptCard.tsx · EbookCard.tsx · BundleCard.tsx
│  └─ checkout/CheckoutForm.tsx · FormatSelector.tsx · PaymentMethod.tsx · MoMoStatus.tsx
```

```
| ├── downloads/DownloadButton.tsx · FormatSwitch.tsx
|   └── admin/ProductForm.tsx · BundleBuilder.tsx · AnalyticsCharts.tsx
└── db/
    └── schema.sql # schema in this report
└── middleware.ts # auth guards, admin role check
└── .env.local # see env block
    └── vercel.json # function config (maxDuration, regions)
```

8 File Storage & Security

Supabase Storage buckets organized by product type, with access control so files are only downloadable by buyers with verified paid orders.

Bucket layout

```
supabase.storage
├─ product-files/ (PRIVATE – server-only access via service role)
│ └─ prompts/ pkg-fre-01/prompts.pdf, prompts.docx
│ └─ ebooks/ ebk-mkt-01/book.pdf, book.epub
│ └─ bundles/ bnd-mkt-01/bundle-v1.zip (pre-built ZIPs)
├─ samples/ (PUBLIC – 3-prompt previews, free chapters)
│ └─ prompts/ · ebooks/
└─ covers/ (PUBLIC – cover images 600×900)
```

Access control rules

Layer	Rule
Bucket privacy	product-files is private: no anon access; only the service-role key (server-side) can read objects. samples/ and covers/ are public.
RLS on tables	products/orders/payments/downloads readable per policy: owner (user_id = auth.uid() or email match for guests) or admin role. Writes: admin only.
Download validation	GET /api/downloads/[id] checks: order.status = paid/fulfilled, order belongs to the requester, format ∈ product.formats. Only then a signed URL is created.
Signed URLs	createSignedUrl(path, 172800) → 48-hour expiry (Supabase supports up to 7 days). URLs contain an HMAC token; expiry is enforced by the storage gateway.
Link revocation	Old links die on expiry; regenerating a link does not invalidate older ones, so TTL is the main control. For refunds, the order is flipped to refunded and download routes reject it.
Uploads	Admin uploads go through server routes with service-role client; files validated (MIME sniff, size ≤ 50 MB, virus scan via simple extension + header check at launch).

9 Storage Usage Projection & Growth Plan

Ebooks are 5–50 MB each; the 1 GB Supabase free tier comfortably covers launch, with a clear ladder before hitting the ceiling.

Item	Count at launch	Avg size	Storage
Prompt packs (Word + PDF)	15	2 MB	30 MB
Ebooks (PDF + EPUB)	5	15 MB (both formats)	75 MB
Bundle ZIPs (pre-built, duplicate content)	3	45 MB	135 MB
Covers + samples + misc	—	—	10 MB
Launch total			≈ 250 MB (25% of the 1 GB free tier)
Wave 2 additions (5–8 products/mo)	~+80 MB/mo		1 GB reached ≈ month 9–10

Growth ladder

Trigger	Action	Cost
Monthly check (free)	Supabase Storage dashboard: track GB used vs 1 GB limit. Set a low-email alert at 80%.	Free
Ebook bloat	Re-export ebooks with compressed images (target ≤ 10 MB per book); regenerate EPUB with tighter assets.	Free
ZIP duplication	Rebuild ZIPs only when component files change; keep one canonical ZIP per bundle version.	Free
At ~80% of 1 GB	Move static files (covers, samples) to a public bucket on Cloudflare R2 (10 GB free, S3-compatible) or keep covers on Supabase and compress more aggressively.	Free
Above 1 GB (growth mode)	Option A: upgrade Supabase Storage (\$0.021/GB-month). Option B: migrate product files to R2 (10 GB free) + Cloudflare signed URLs via a Worker; keep Supabase for DB/auth.	\$0–2/mo
Commercial volumes (1000+ orders/mo)	Move files behind Cloudflare R2 + custom CDN rules; consider Vercel Pro only if CPU/bw needs grow.	~\$5–25/mo

10 Environment Variables

Secrets live in Vercel project settings (never in git). Client-safe keys use NEXT_PUBLIC_ prefix.

```
# .env.local (never commit – Vercel project settings hold prod values)
NEXT_PUBLIC_SITE_URL=https://your-store.vercel.app

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi... # server-only

# Paystack (Ghana)
PAYSTACK_SECRET_KEY=sk_live...
PAYSTACK_PUBLIC_KEY=pk_live...
PAYSTACK_WEBHOOK_SECRET=whsec...

# Flutterwave (fallback gateway)
FLUTTERWAVE_SECRET_KEY=FLWSECK_TEST-...
FLUTTERWAVE_WEBHOOK_SECRET=FLWSECK-...

# Which gateway is active for new checkouts
PAYMENT_GATEWAY=paystack # | flutterwave

# Email (Resend)
RESEND_API_KEY=re...
FROM_EMAIL=sales@your-store.vercel.app

# SMS (optional, post-launch)
TERMII_API_KEY=... (or ARKESEL_KEY=...)

# Storage
DOWNLOAD_LINK_TTL_SECONDS=172800 # 48 hours
BUNDLE_ZIP_MAX_MB=150
MAX_FILE_SIZE_MB=50
```

11 Payment Resilience & Fallback Plan

Gateway downtime and failed transactions are handled without losing orders or trust.

Scenario	Detection	Fallback behavior
Paystack API down	Timeout/5xx on /transaction/initialize; health check on gateway status page.	Checkout shows “Payments temporarily unavailable, try again in a few minutes”; no order is created for failed init; status banner on store.
Gateway webhook delayed	Payment authorized but order still pending after 60 s.	Confirmation page polls GET /api/orders/[id] every 3 s; buyer can tap “I’ve approved — check status” which calls /verify.
Webhook missed entirely	Paystack retries for 3 days; if still absent, order stays pending.	Admin /api/admin/orders lists pending orders with a “Verify with gateway” button (GET /transaction/verify/:reference) that reconciles status and fulfills.
Failed / declined charge	charge.failed webhook or verify returns failed.	Order marked failed; buyer sees “Payment failed — try again or choose another method”; cart preserved.
Fallback gateway switch	Flag PAYMENT_GATEWAY=flutterwave; both integrations share the same orders/payments schema.	New checkouts use Flutterwave; existing pending orders still resolvable via Paystack verify.
Chargeback / duplicate charge	Duplicate reference or user dispute.	Idempotency on reference prevents double fulfillment; duplicates auto-refunded; admin refund flow updates payments + orders to refunded and revokes download access.

12 Zero-Cost Launch Checklist

Every service confirmed on a free tier — the only variable cost is the ~1.95% per successful mobile-money transaction.

Service	Free tier used	Launch cost
Vercel (Next.js + API routes + webhooks)	Hobby — 1M invocations, 100 GB transfer, 60 s functions	GH¢ 0
Supabase — Postgres + Auth	500 MB DB, 50K MAU, email/OTP auth	GH¢ 0
Supabase — Storage	1 GB objects, 5 GB egress/mo	GH¢ 0
Paystack (Ghana)	Free integration; ~1.95% per successful txn (capped GHS 100)	Per-sale only
Flutterwave (fallback)	Free integration; ~1–2.9% per txn	Per-sale only
Resend (transactional email)	3,000 emails/mo	GH¢ 0
Domain	your-store.vercel.app subdomain	GH¢ 0
SMS (optional)	Deferred to post-launch (Termii/Arkesal pay-as-you-go)	GH¢ 0 at launch
Analytics/monitoring	Vercel dashboard + Supabase logs	GH¢ 0
Total monthly fixed cost at launch		GH¢ 0 (variable: ~1.95% per sale)

Go-live gates: Paystack live keys issued for a Ghana-registered business (individual accounts can start with test keys + a limited live mode) · webhook URL set in the Paystack dashboard · signed-URL TTL set to 172800 s · at least one paid test order fulfilled end-to-end on both MTN and Telecel networks · storage usage logged in a monthly reminder.